

词法分析器实验报告

程宣赫 2023202250

2026 年 3 月 24 日

1 实验目标

本实验的目标是结合 SysY 语言的词法规则，使用 flex 工具设计并实现一个词法分析器。实验要求程序能够对输入的 SysY 源程序进行扫描，识别其中的关键字、标识符、整型常量、算符和界符等单词符号，并输出相应的类别及位置信息。

2 实验环境

本实验在以下环境中完成：

- 操作系统：Linux (Ubuntu)
- 编译工具：flex 和 gcc
- 编辑器：VS Code / Overleaf
- 运行方式：命令行编译与执行

3 实验原理与设计思路

3.1 SysY 文法分析

从给定的 SysY 文法可以看出，整个程序的开始符号是 `CompUnit`。这说明一个 SysY 程序整体上是由若干个声明和函数定义组成的，也就是说，程序最外层主要包括常量声明、变量声明和函数定义这几部分。

再往下，SysY 支持的基本类型比较简单，只有 `int`，函数返回类型则有 `int` 和 `void` 两种。语句部分支持赋值语句、表达式语句、代码块、条件语句 `if-else`、循环语句 `while`，以及 `break`、`continue`、`return` 等跳转语句。除此之外，文法中还包含数组访问、函数调用以及算术、关系和逻辑表达式等内容。

从词法分析的角度来看，文法里出现的这些终结符正是词法分析器需要识别的对象。根据语法规则，可以进一步整理出词法分析器需要处理的几类基本单词符号。首先是关键字，包括 `const`、`int`、`void`、`if`、`else`、`while`、`break`、`continue` 和 `return`；其次是标识符，对应文法中的 `Ident`；再次是整型常量，对应文法中的 `IntConst`。此外，文法中还大量使用了各种算符和界符，例如赋值运算符、加减乘除模运算符、关系运算符、逻辑运算符，以及圆括号、方括号、花括号、逗号和分号等。最后是注释部分。这些内容共同构成了词法分析阶段需要识别的基本 token 类型。

3.2 词法分析器设计思路

本实验的实现直接围绕 `flex` 文件的基本结构展开。结合 `SysY` 文法，可以先确定词法分析器需要识别的主要单词类别为关键字、标识符、常量、算符、界符以及其他字符，对应实验要求中的 K、I、C、O、D、T 六类。因此，在设计词法分析器时，需要把文法中出现的这些终结符和词法成分进一步转化为 `flex` 中的正则表达式与匹配规则。

在声明与辅助定义部分，主要完成头文件引入、行列号变量定义以及若干正则表达式的抽象。例如，根据文法中 `Ident` 和 `IntConst` 的形式，分别定义数字、字母、标识符、十进制整数、八进制整数和十六进制整数等规则，为后续识别提供基础。对于多行注释，还引入了独占状态，用于处理注释内部内容。

在识别规则部分，则按照文法中涉及的词法成分依次设计匹配规则。首先处理空白符、换行符和注释，因为这些内容虽然不属于最终输出的 token，但会影响扫描位置；随后根据文法中给出的保留字，设置关键字识别规则；根据 `Ident` 和 `IntConst` 的形式，分别设置标识符和整型常量的识别规则；再结合文法中的运算符和界符，补充对应的匹配规则。最后，对于前面规则都无法匹配的字符，统一归入其他类型，以保证输入串中的所有内容都能被处理。

我们的整体设计思路实际上就是把 `SysY` 文法中已经给出的关键字、标识符、常量、算符和界符，按照 `flex` 的文件结构逐步映射为声明、正则表达式和识别规则，从而完成词法分析器的构建。

4 代码实现思路

本实验的代码实现主要围绕词法分析过程中的几个关键问题展开，包括位置信息维护、注释与空白处理、各类单词符号的识别以及异常输入的处理。整体上，程序按照输入扫描的顺序逐步完成 token 的匹配与输出。

4.1 位置维护与辅助函数

为了满足实验中输出行号和列号的要求，程序设置了当前行号和列号两个全局变量，用于记录扫描过程中的当前位置。在识别出某个 token 时，先保存其起始位置，再输出对应的类别信息，最后根据匹配到的字符串长度更新当前位置。对于这一过程，程序通过辅助函数统一完成位置更新和 token 输出，从而保证不同规则下的位置处理方式保持一致。

4.2 注释与空白处理

空白符、换行符和注释虽然不属于最终需要输出的 token，但会直接影响位置信息，因此在实现中优先处理。其中，空格和换行主要用于更新列号和行号；单行注释可直接匹配到行末；多行注释由于可能跨越多行，且内部可能包含各种普通字符，因此采用独占状态的方式处理。进入注释状态后，程序只继续扫描注释内容本身，直到遇到结束符为止，从而避免将注释内部字符误识别为正常代码成分。

4.3 各类单词符号的识别实现

在识别规则设计中，程序根据 SysY 文法中提取出的词法成分分别实现了关键字、标识符、整型常量、算符和界符的识别。关键字直接按固定字符串进行匹配；标识符根据“首字符为字母或下划线、后续字符可为字母、数字或下划线”的特点进行识别；整型常量则进一步分为十进制、八进制和十六进制三种形式。对于算符和界符，程序按照文法中出现的字符符号逐项设置对应规则。

在规则顺序上，程序特别注意了不同模式之间的优先关系。例如，关键字规则放在标识符规则之前，以避免保留字被误识别为普通标识符；双字符算符放在单字符算符之前，以避免出现错误拆分。通过这种方式，可以保证主要 token 的识别结果与 SysY 文法保持一致。

4.4 兜底与错误处理

对于前述规则都无法匹配的字符，程序通过最后的兜底规则将其统一归入其他类型，从而保证输入字符流中的所有内容都能得到处理。此外，对于部分明显不符合要求的输入形式，也通过专门规则进行识别，以提高词法分析结果的完整性和稳定性。通过这种“前面优先识别合法 token，最后统一处理异常输入”的方式，程序能够较好地完成整个扫描过程。

5 实验结果分析

在测试过程中，分别使用了 example 文件中的 .sy 文件与 .out 文件对应检查，以及 test-cases 文件夹中的多个 .sv 测试文件来验证词法分析器的正确性：